



Amazon Elastic Compute Cloud (EC2)

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

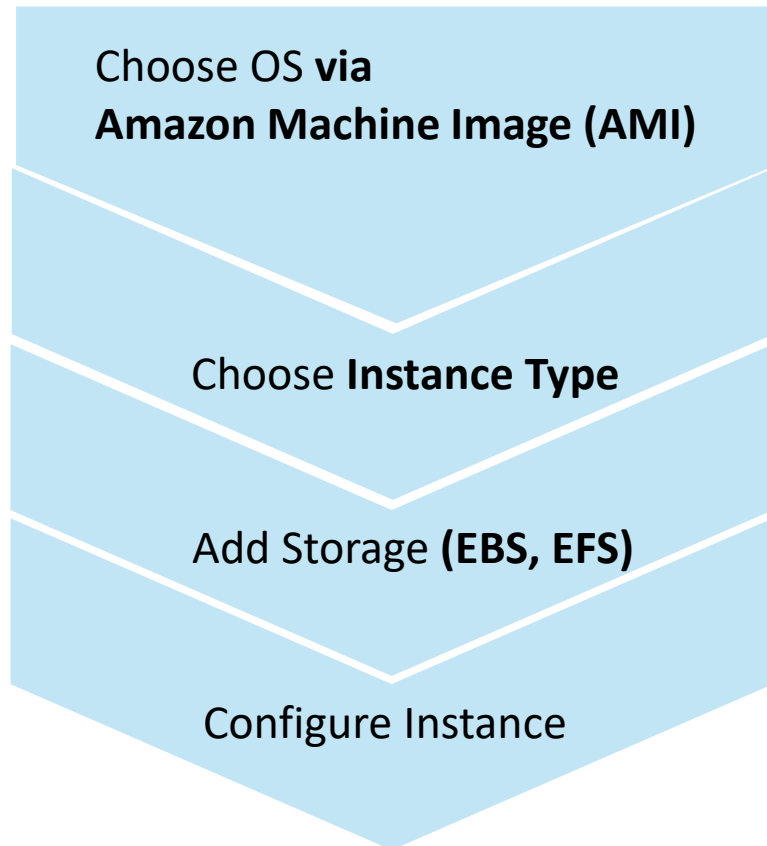
Introduction to EC2



Elastic Compute Cloud (EC2) is a **highly configurable virtual server**.

EC2 is resizable **compute capacity**. It takes **minutes** to launch new instances.

Anything and everything on AWS uses EC2 Instance underneath.



Red Hat



ubuntu



Amazon Linux



t2.nano

\$0.0065/hour (\$4.75/month)

1 vCPU 0.5GB Mem

C4.8xlarge

\$1.591/hour (\$1161.43/month)

36 vCPU 60GB Mem 10 Gigabit performance

SSD HDD Virtual Magnetic Tape Multiple Volumes

Security Groups, Key Pairs, UserData, IAM Roles, Placement Groups



Cloud-Init

Cloud-Init is a multi-distribution method **cloud instance initialization**.

It is used across public cloud providers, provisioning systems for private cloud systems, and bare-metal installations.

What is Cloud Instance Initialization?

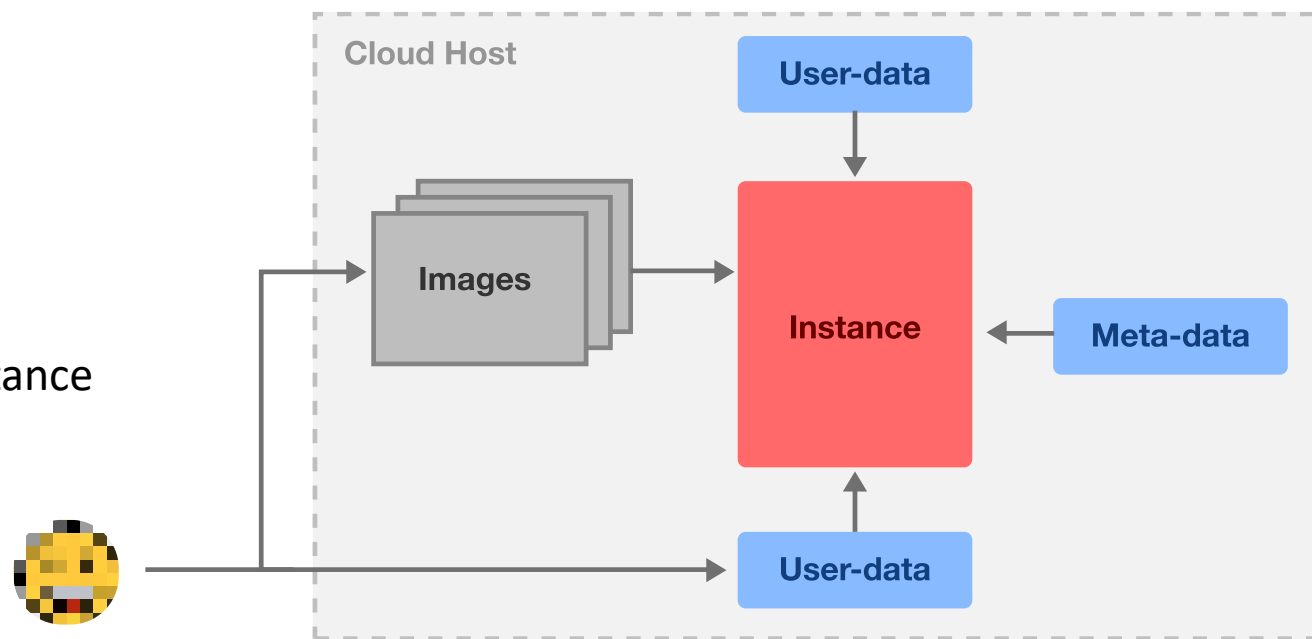
The process of preparing an instance with configuration data for the operation system and runtime environment.

Cloud instances are initialized from a disk image and instance data:

- Meta-data
- **User-data**
- Vendor-data

User Data is a script that you want to run when an instance first boots up. (eg. Install Apache web server)

AWS EC2 supports Cloud-Init across most Linux distributions



EC2 – User Data

You can provide a script to EC2 UserData to have Cloud Init automatically run on first boot.

You can provide a **bash script**



```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd.service
systemctl enable httpd.service
echo "<html><h1>Hello Cloud!</h1></html>" > /var/www/html/index.html
```

You can provide a **cloud config yaml file**



```
#cloud-config
package_upgrade: true
packages:
  - httpd
runcmd:
  - [ systemctl, start, httpd.service ]
  - [ systemctl, enable, httpd.service ]
  - [ sh, -c, "echo '<html><body><h1>Hello Cloud!</h1></body></html>' > /var/www/html/index.html" ]
```



Scripts must be base64 when directly using the API, The AWS CLI and Console will automatically encode to base64

EC2 – User Data

Via the AWS CLI, you can **pass a file**



```
aws ec2 run-instances \  
  --image-id ami-0abcdef1234567890 \  
  --count 1 \  
  --instance-type t2.micro \  
  --security-group-ids sg-1234567890abcdef0 \  
  --subnet-id subnet-12345678 \  
  --user-data file://path/to/your/userdata-script.sh
```

Or you can provide the **script inline**



```
aws ec2 run-instances \  
  --image-id ami-0abcdef1234567890 \  
  --count 1 \  
  --instance-type t2.micro \  
  --security-group-ids sg-1234567890abcdef0 \  
  --subnet-id subnet-12345678 \  
  --user-data '#!/bin/bash  
yum update -y  
yum install -y httpd  
systemctl start httpd  
systemctl enable httpd  
echo "<html><h1>Welcome to Apache Server on EC2</h1>  
</html>" > /var/www/html/index.html'
```

Using EC2 Metadata, you can get the script via **wget** or **curl** at the following <http://169.254.169.254/latest/user-data>

EC2 – Meta Data

From within your EC2 instance, you can access EC2 metadata information from the Metadata Service (MDS) via a special endpoint.

There are two versions of Metadata Service

- Instance Metadata Service Version 1 (IMDSv1) – a request/response method
- Instance Metadata Service Version 2 (IMDSv2) – a session-oriented method



IMDSv2 was implemented after an exploit of IMDSv1 was discovered. IMDSv2 adds defense in depths attacks against open firewalls, reverse proxies and SSRF vulnerabilities

The endpoint address:

- IPv4 — <http://169.254.169.254/latest/meta-data/>
- IPv6 — <http://fd00:ec2::254/latest/meta-data/>

EC2 – Meta Data

IMDSv1 using curl

```
curl http://169.254.169.254/latest/meta-data
```

IMDSv2 using curl

```
TOKEN=`curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600"` \
&& curl -H "X-aws-ec2-metadata-token: $TOKEN" -v http://169.254.169.254/latest/meta-data/
```

You could also use wget in place of curl, depending on what you have installed on your OS

```
wget -q0 - http://169.254.169.254/latest/meta-data/
```

EC2 – Meta Data

Instance Metadata is grouped into categories. There are over 60+ categories.

ami-id
ami-launch-index
ami-manifest-path
ancestor-ami-ids
autoscaling/target-lifecycle-state
block-device-mapping/ami
block-device-mapping/ebsN
block-device-mapping/ephemeralN
block-device-mapping/root
block-device-mapping/swap
elastic-gpus/associations/elastic-gpu-id
elastic-inference/associations/eia-id
events/maintenance/history
...

```
# Step 1: Fetch the session token and store it
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600" \
--silent)

# Step 2: Use the token to fetch the security groups
curl -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/security-groups \
--silent
```

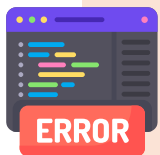
You simply **add to end of the endpoint** the category

EC2 – Meta Data

There are configuration options you can configure around Meta Data:

- You can **enforce the use of tokens (IMDSv2)** →
- You can turn off the endpoint altogether
- You can specify the amount of network hops allowed

```
aws ec2 modify-instance-metadata-options \  
  --instance-id i-1234567890abcdef0 \  
  --http-tokens required \  
  --http-endpoint enabled \  
  --http-put-response-hop-limit 1 \  
  --region your-instance-region
```



If you receive a 401 when attempting the use without a token (IMDSv2) it's likely that a token is required, so you must use IMDSv2

EC2 – Instance Types

The Instance type naming convention is as follows:

Instance Family

The intended workflow the instance-type is designed to meet.

Processor family

A specific type of processor

Instance size

The amount of available virtual resources eg. vCPU and RAM

Instance generation

*The versions of the family offerings.
Affects the version of hardware. Newer means better.*

Additional capability

Extra features this instance-type has.
Eg. Extra Storage.

c7gn.xlarge



Some parts of the instance type may be omitted.

Eg. t3.micro does not contain Processor Family or Additional Capacity

EC2 Instance Families

What are Instance Families?

Instance families are different combinations of CPU, Memory, Storage, and Networking capacity.

Instance families allow you to choose the appropriate combination of capacities to meet your application's unique requirements.

Different instance families are different because of the varying hardware used to give them their unique properties.

Commonly, instance families are called "Instance Types", but an instance type is a combination of size and family.

General Purpose

A1 T2 T3 T3a T4g M4 M5 M5a M5n M6zn M6g M6i Mac

balance of compute, memory, and networking resources

Use-cases *web servers and code repositories*

Compute Optimized

C5 C4 Cba C5n C6g C6gn

Ideal for compute-bound applications that benefit from high-performance processor

Use-cases *scientific modeling, dedicated gaming servers, and server engines*

Memory Optimized

R4 R5 R5a R5b R5n X1 X1e High Memory z1d

fast performance for workloads that process large data sets in memory.

Use-cases *in-memory caches, in-memory databases, real-time big data analytics*

Accelerated Optimized

P2 P3 P4 G3 G4ad G4dn F1 Inf1 VT1

hardware accelerators, or co-processors

Use-cases *Machine learning, computational finance, seismic analysis, speech recognition*

Storage Optimized

I3 I3en D2 D3 D3en H1

high, sequential read and write access to very large data sets on local storage

Use-cases *NoSQL, in-memory or transactional databases, data warehousing*

EC2 – Instance Family

Both T and M families are **General Purpose**.



The M Family offers an even balance of compute, memory, and networking



The T Family is the cheapest family.
Great for variable workloads



C – Compute optimized

D – Dense storage

F – FPGA

G – Graphics intensive

Hpc – High performance computing

I – Storage optimized

Im – Storage optimized with a 1:4 (vCPU:Memory)

Is – Storage optimized with a 1:6 (vCPU:Memory)

Inf – AWS Inferentia

M – General purpose

Mac – macOS

P – GPU accelerated

R – Memory optimized

T – Burstable performance

Trn – AWS Trainium

U – High memory

VT – Video transcoding

X – Memory intensive

EC2 – Processors

AWS underlying instance can have **access to a variety of different processors** to meet specific cloud workload needs.



Intel Xeon Processors

Similar to Intel Desktop CPUs but with advanced capabilities.



Intel Habana Gaudi Processors Specialized processors for machine learning workloads



AMD EPYC Processors

Alternatives to Intel-based instances.
Potential cost saving over Intel processors



NVIDIA GPUs

Graphic intensive workloads.
Often used in Machine Learning.

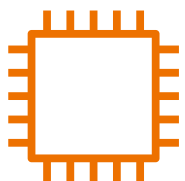


Intel FPGAs

Intel's offering for field-programmable gate arrays for workloads that benefit from custom hardware accelerations

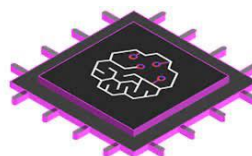
Xilinx (AMD)

AMD's offering for field-programmable gate arrays for workloads that benefit from custom hardware accelerations



AWS Graviton Processors

Custom-built by AWS using ARM architecture



AWS Inferentia

A processor designed to deliver high-performance ML inference at low cost

AWS Trainium —A processor designed to train ML models at low cost.

EC2 – Burstable Instances

Burstable instances allow workloads to handle bursts of higher CPU utilizations **for very short durations**.

This allows AWS customers to save money overall since they do not need to upgrade their instance based on highest peak usage.

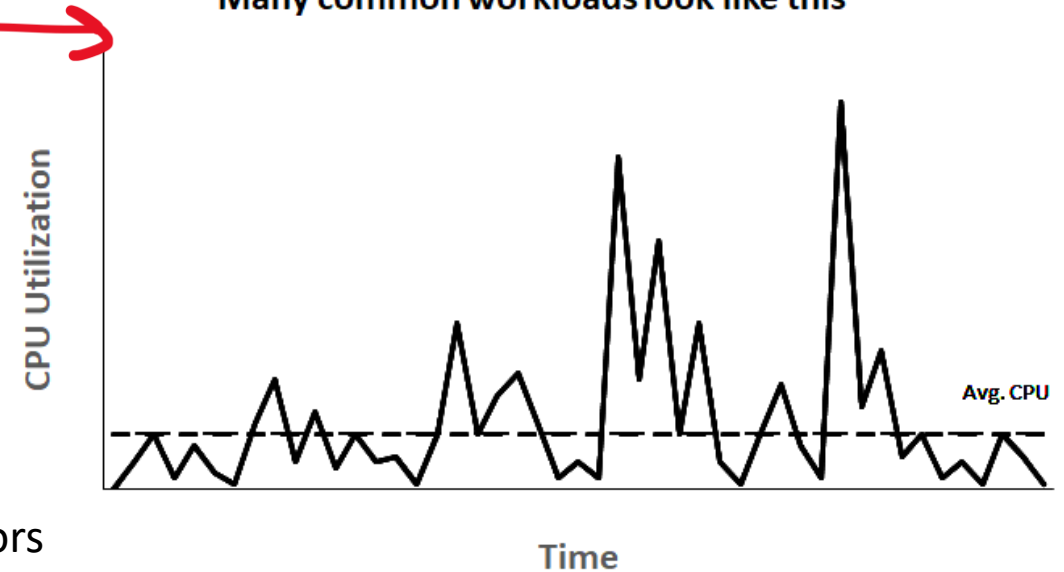
T4g — Graviton (ARM), cheaper than t4g

T3a — AMD EPYC, cheaper than T3

T3 — Intel Xeon Scalable, best overall

T2 — Intel Xeon, free tier due to previous generation processors

Many common workloads look like this



Standard Mode

(Default): Provides a baseline level of CPU performance with the ability to burst above the baseline using accumulated CPU credits, suitable for workloads with variable CPU usage.

Unlimited Mode:

Allows an instance to sustain high CPU performance for any period whenever required, exceeding the baseline and accumulated CPU credits, with additional charges applied for the extra CPU usage beyond the accumulated credits.

EC2 Instance Sizes

Each instance type includes one or more instance sizes, allowing you to scale your resources to the requirements of your target workload.

EC2 Instance Sizes **generally double** in price, and key attributes

Name	vCPU	RAM (GiB)	On-Demand per hour	On-Demand per month
t2.small	1	2	\$0.023	\$16.79
t2.medium	2	4	\$0.0464	\$33.87
t2.large	2	8	\$0.0928	\$67.74
t2.xlarge	4	16	\$0.1856	\$135.48

EC2 – Instance Profile

EC2 Instance Profile is a **reference to an IAM role** that an EC2 instance assumes when it starts



EC2 Instance Profile allows EC2 instances to access AWS services **without** storing long-term credentials (e.g., AWS Access Key and Secret)

- An **EC2 Instance Profile** can be associated **at launch or while running**
 - If no previous EC2 Instance Profile was attached, a **hard reboot is required** for the role to be assumed.
- Only one IAM role can be associated with an Instance Profile
- Changing roles is not instantaneous due to consistency mechanisms
 - To change immediately, you must **disassociate/reassociate** the profile or **hard reboot the instance**



When you select an IAM role during instance launch, AWS automatically creates an instance profile. Instance Profiles are **not easily visible in the AWS Console**

EC2 – Instance Profile

```
aws iam create-instance-profile \  
--instance-profile-name MyInstanceProfile
```

Creates an Instance Profile

```
aws iam add-role-to-instance-profile \  
--instance-profile-name MyInstanceProfile \  
--role-name MyRole
```

Adds an IAM role to the Instance Profile

```
aws ec2 associate-iam-instance-profile \  
--instance-id i-1234567890abcdef0 \  
--iam-instance-profile Name=MyInstanceProfile
```

Associating the Instance Profile to the E2C Instance

```
aws iam list-instance-profiles
```

Lists all EC2 Instance Profile

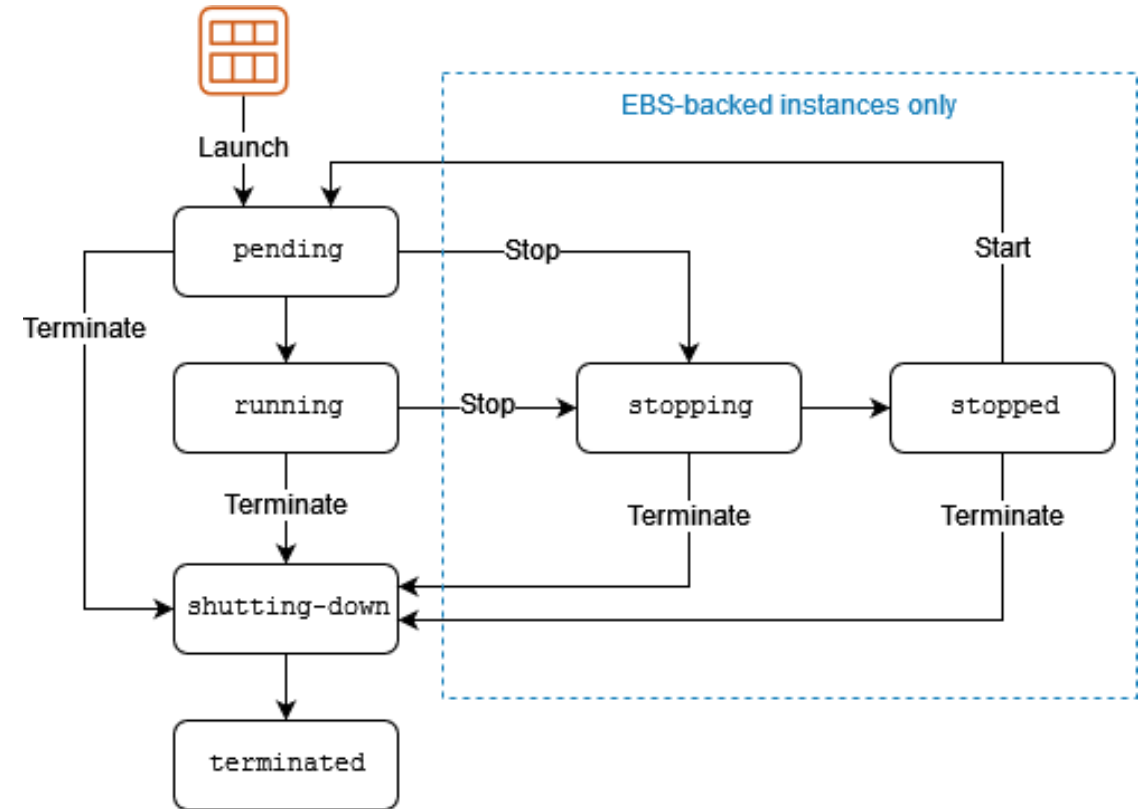
```
aws iam get-instance-profile \  
--instance-profile-name MyInstanceProfile
```

Gets details about a specific EC2 Instance Profile

EC2 – Instance Lifecycle

Actions:

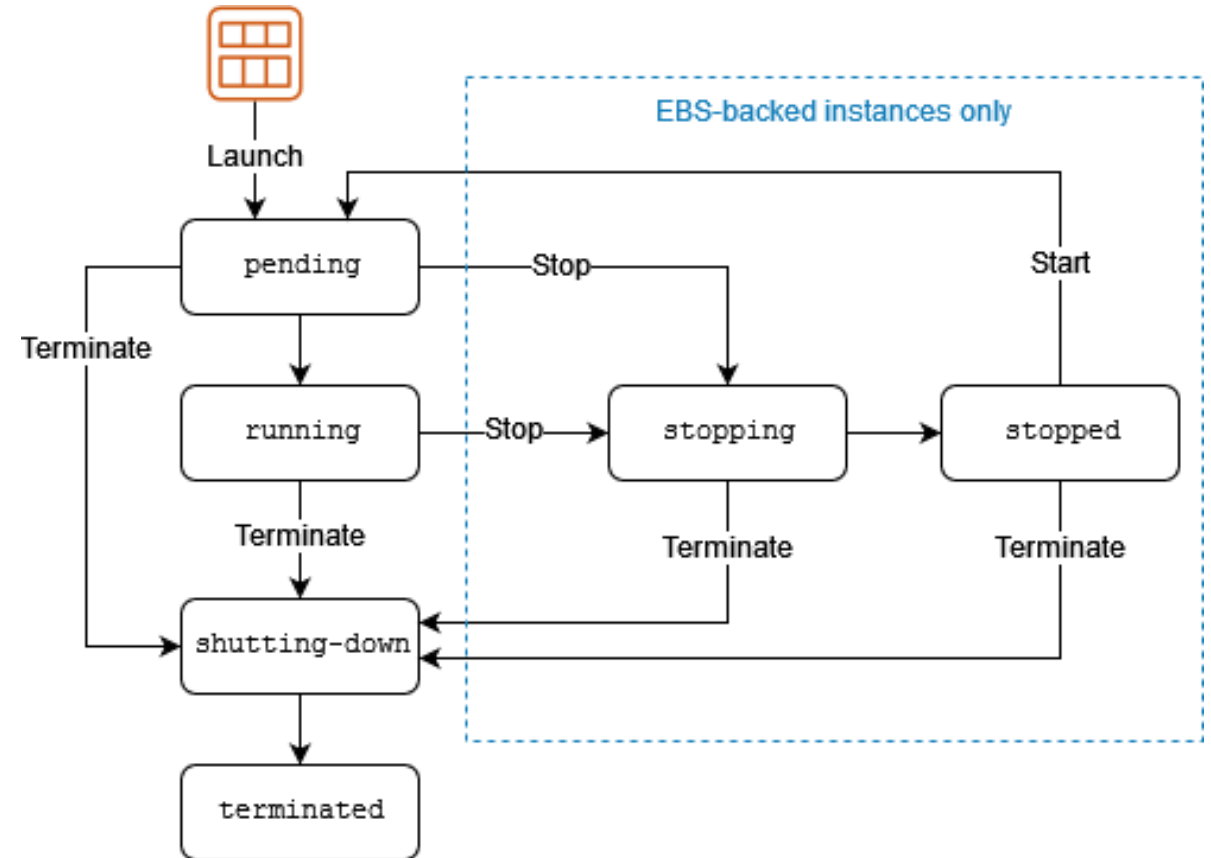
- **Launch** — Create and start an EC2 instance based on AMI
- **Stop** — Turn off but not delete the current EC2 instance
- **Start** — Turn back on a previously stopped EC2 instance
- **Terminate** — Delete the EC2 instance
- **Reboot** — Performs a Soft reboot
- **Retire** — notifies when an instance is scheduled for retirement due to hardware failure or end of life; it must be replaced or migrated.
- **Recover** — Automatically recovers a failed instance on new hardware if enabled, keeping the instance ID and other configurations.



EC2 – Instance Lifecycle

States:

- **pending** — The instance is preparing to enter the running state. An instance enters the pending state when it is launched or when it is started after being in the stopped state.
- **running** — The instance is running and ready for use.
- **stopping** — The instance is preparing to be stopped.
- **stopped** — The instance is shut down and cannot be used. The instance can be started at any time.
- **shutting-down** — The instance is preparing to be terminated.
- **terminated** — The instance has been permanently deleted and cannot be started.



EC2 – Instance Lifecycle

Change Termination Protection

Prevent the instance from being terminated
(Default is off)

```
aws ec2 modify-instance-attribute \  
--instance-id i-1234567890abcdef0 \  
--disable-api-termination
```

Change Stop Protection

Prevent the instance from being stopped
(Default is off)

```
aws ec2 modify-instance-attribute \  
--instance-id i-1234567890abcdef0 \  
--disable-api-stop
```

Shutdown Behavior

What should happen if the instance is shut down to either Stop or Terminate
(Default is Stop)

```
aws ec2 modify-instance-attribute \  
--instance-id i-1234567890abcdef0 \  
--instance-initiated-shutdown-behavior terminate
```

Auto-Recovery Behavior

Whether to automatically recover an instance when a system status check failure occurs
(Default is enabled)

```
aws ec2 modify-instance-maintenance-options \  
--instance-id i-0abcdef1234567890 \  
--auto-recovery disabled
```

EC2 – Source and Destination Checks

The **EC2 source/destination check** (when enabled) restricts an instance to only send and receive traffic where it is the source or the destination.



```
aws ec2 modify-instance-attribute \  
--instance-id i-1234567890abcdef0 \  
--no-source-dest-check
```

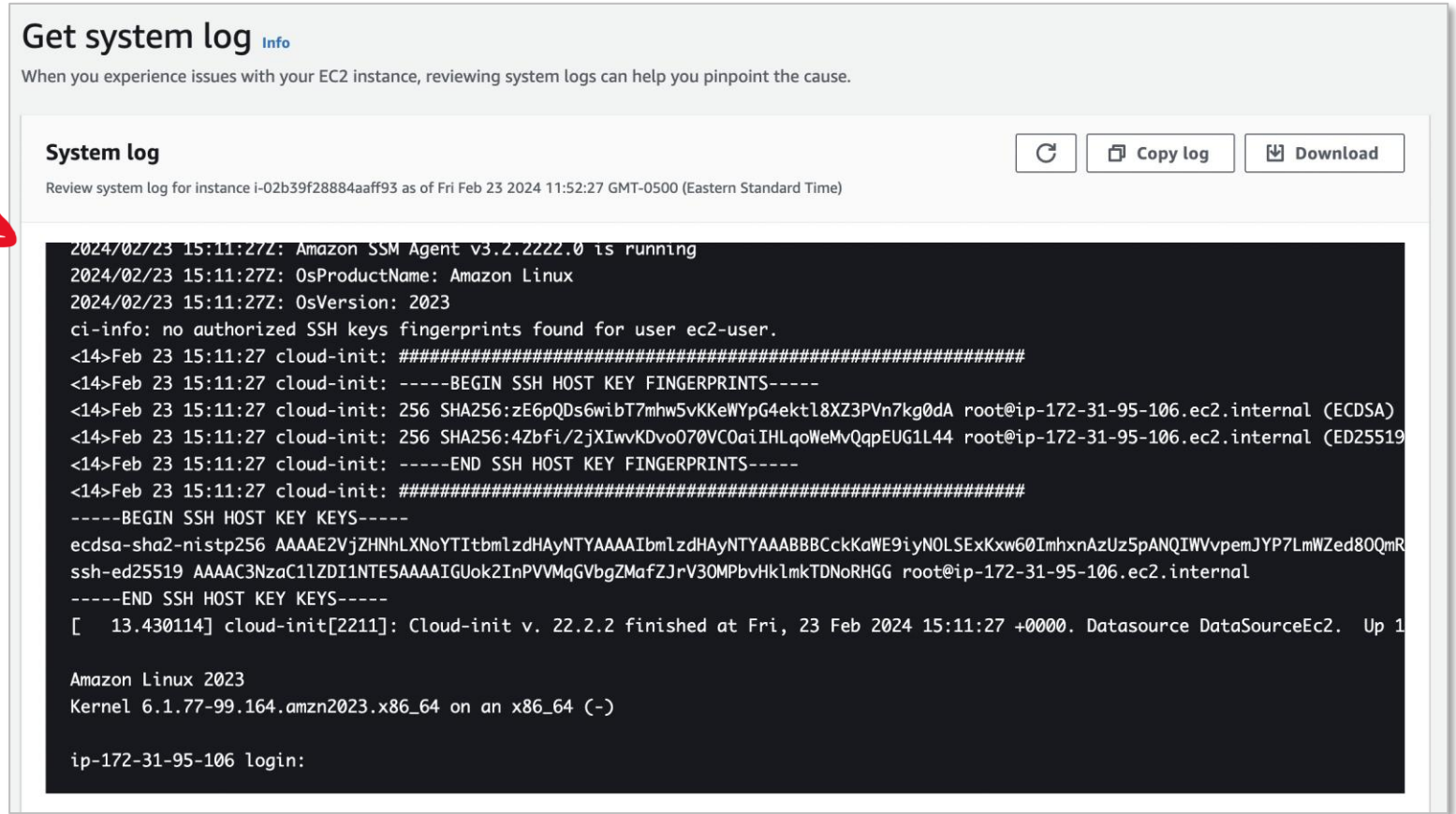
Disabling source and destination check is useful in cases where you need an EC2 instance to pass through it, like utilizing the instance for Network Address Translation (NAT)

EC2 – System Log

EC2 Management Console allows you to **observe the system log** for the EC2 Instance.

This is useful if you're trying to troubleshoot an instance on boot to see if anything is wrong.

Some Marketplace and Community AMIs will write the default username and password for software in the System Log for you to initially login.



Get system log [Info](#)

When you experience issues with your EC2 instance, reviewing system logs can help you pinpoint the cause.

System log 🔄 📄 Copy log 📄 Download

Review system log for instance i-02b39f28884aaff93 as of Fri Feb 23 2024 11:52:27 GMT-0500 (Eastern Standard Time)

```
2024/02/23 15:11:27Z: Amazon SSM Agent v3.2.2222.0 is running
2024/02/23 15:11:27Z: OsProductName: Amazon Linux
2024/02/23 15:11:27Z: OsVersion: 2023
ci-info: no authorized SSH keys fingerprints found for user ec2-user.
<14>Feb 23 15:11:27 cloud-init: #####
<14>Feb 23 15:11:27 cloud-init: -----BEGIN SSH HOST KEY FINGERPRINTS-----
<14>Feb 23 15:11:27 cloud-init: 256 SHA256:zE6pQDs6wibT7mhw5vKKeWYpG4ekt18XZ3PVn7kg0dA root@ip-172-31-95-106.ec2.internal (ECDSA)
<14>Feb 23 15:11:27 cloud-init: 256 SHA256:4Zbfi/2jXIwvKDvo070VC0aiIHLqoWeMvQqpEUG1L44 root@ip-172-31-95-106.ec2.internal (ED25519)
<14>Feb 23 15:11:27 cloud-init: -----END SSH HOST KEY FINGERPRINTS-----
<14>Feb 23 15:11:27 cloud-init: #####
-----BEGIN SSH HOST KEY KEYS-----
ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBcKkKaWE9iyNOLSExKxw60ImhxnAzUz5pANQIWVvpemJYP7LmWZed80QmR
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGUok2InPVVMqGVbgZMafZJrV30MPbvHkLmkTDNoRHGG root@ip-172-31-95-106.ec2.internal
-----END SSH HOST KEY KEYS-----
[ 13.430114] cloud-init[2211]: Cloud-init v. 22.2.2 finished at Fri, 23 Feb 2024 15:11:27 +0000. Datasource DataSourceEc2. Up 1
Amazon Linux 2023
Kernel 6.1.77-99.164.amzn2023.x86_64 on an x86_64 (-)

ip-172-31-95-106 login:
```



Logs can be delivered to CloudWatch Logs installing the CloudWatch Unified agent and can be accessed through a CloudWatch Log group of the same name as the instance.

EC2 – System Log

Logs that would be collected from an EC2 Instance if CloudWatch Unified Agent is running and the EC2 Instance is permitted to write to CloudWatch Logs

/var/log/amazon/ssm/amazon-ssm-agent.log /var/log/amazon/ssm/errors.log
/var/log/audit/audit.log
/var/log/cloud-init-output.log
/var/log/cfn-init.log
/var/log/cfn-init-cmd.log
/var/log/cloud-init.log (AL1 / AL2 only)
/var/log/cron
/var/log/maillog
/var/log/messages
/var/log/secure
/var/log/spooler
/var/log/yum.log
/var/log/aws/ams/bootstrap.log
/var/log/aws/ams/build.log
/var/log/syslog
/var/log/dpkg.log
/var/log/auth.log
/var/log/zypper.log

EC2 – Instance Console Screenshot

Instance Console Screenshot will take a **screenshot of the current state** of the instance.

This is useful for troubleshooting booting issues when you cannot access the instance via SSH or RDP

You can use the **AWS CLI** to grab the screenshot

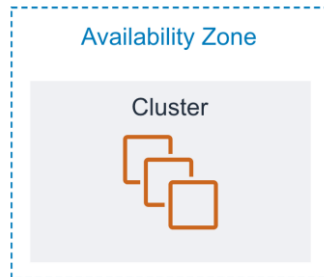
```
aws ec2 get-console-screenshot
```



If something went wrong, you might see something other than a login screen

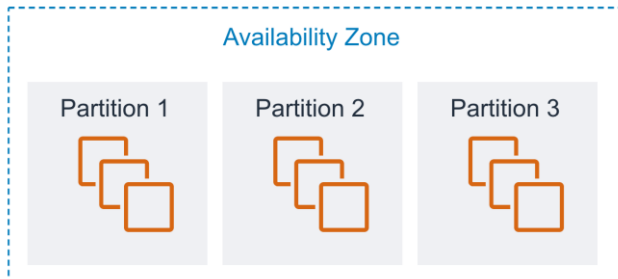
EC2 – Placement Groups

Placement Groups determine **the logical placement** of EC2 instances to optimize for **communication, performance, or durability**. Placement groups are **free** to use.



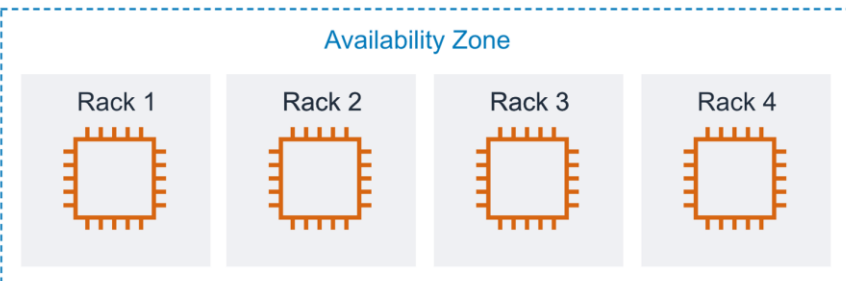
Cluster

- Packs instances in a single **Availability Zone (AZ)**
- Optimized for **low-latency, high throughput network performance**
- Best for **High-Performance Computing (HPC)** applications
- Cannot span across multiple AZs



Partition

- Spreads instances across logical partitions (each on isolated racks)
- Prevents failure impact across partitions
- Instances in the same partition do not share racks
- **Can span multiple Availability Zones (Multi-AZ)**
- Ideal for **large distributed workloads** (eg., Hadoop, Cassandra, Kafka)



Spread

- Each instance placed on a **separate rack**
- Used for high availability and fault tolerance
- Maximum of **7 instances** per AZ per spread group
- Can span multiple AZs

EC2 – Connect

SSH Client

- Connect to your local machine via an **SSH connection** using public and private keys
- Generate key pairs on **AWS** and download the public key
- **Port 22** must be open in the **Security Group** for connectivity

EC2 Instance Connect

Short-lived **SSH keys** managed via **IAM policies**. Works only with **Linux** and not all instances

Sessions Manager

- Connect to **Linux** or **Windows** machines via a reverse connection.
- Windows will log into PowerShell, Linux will log into Bash Shell
- No need to open ports. Managed through **IAM**
- Supports **audit trails** for logins

Fleet Manager Remote Desktop

Connect to **Windows machines** using **RDP** all within the browser

EC2 Serial console

Direct **serial connection** for troubleshooting underlying hardware issues

EC2 – Hostnames

A hostname is a unique name in your network to identify a machine via DNS.

The hostname format will vary based on what compute is being used.



```
hostname  
> ip-172-31-87-245.ec2.internal
```

```
sudo vi /etc/cloud/cloud.cfg  
# preserve_hostname: true
```

Ensure your cloud init configuration will preserve hostname changes

```
sudo hostnamectl set-hostname webserver.mydomain.com  
sudo reboot
```

Change the hostname and reboot for the name to take affect

Changing the hostname could be necessary in specific use cases where software is expecting a very specific name. When building micro-services, such as using Service Meshes, you might have to change hostnames.

EC2 – Hostnames

There are two hostname types for the guest OS hostname

IP Name

legacy naming scheme based on the private IP address.

Used when you launch an IPV4 only EC2 instance.

US-East-1

- private-ipv4-address.ec2.internal
- **ip-10-24-34-0.ec2.internal**

Other Regions

- private-ipv4-address.region.compute.internal
- **ip-10-24-34-0.ca-central-1.compute.internal**

Resource Name

EC2 instance ID is included in the hostname of the instance.

Used when you launch IPV6 only EC2 Instance.

If dual stack, you can choose.

- ec2-instance-id.ec2.internal
- **i-0123456789abcdef.ec2.internal**

- private-ipv4-address.region.compute.internal
- **ip-10-24-34-0.ca-central-1.compute.internal**

EC2 – Default User Name

The default **user name** for an operating system (OS) managed by AWS varies by AMI (Amazon Machine Image). This is important for **connecting via SSH or AWS Systems Manager Session Manager**.

- **Amazon Linux AMI** ec2-user
- **CentOS AMI** centos or ec2-user
- **Debian AMI** admin
- **Fedora AMI** ec2-user
- **RHEL AMI** ec2-user or root
- **SUSE AM** ec2-user or root
- **Ubuntu AMI** ubuntu
- **Oracle Linux AMI** ec2-user
- **Bitnami AMI** bitnami
- Otherwise, check with the AMI provider.

When connecting via Sessions Manager, your session starts as the **ssm-user**. To switch to the default OS user:



```
whoami  
> ssmuser  
sudo su - ec2-user  
> ec2-user
```

EC2 – Amazon Linux

Amazon Linux is AWS's managed Linux distribution is based on CentOS and Fedora, which in turn is based on Red Hat Linux (RHEL)

There are multiple versions of Amazon Linux

- Amazon Linux (AL1) (end of life Dec 2023)
- Amazon Linux 2 (AL2)
- **Amazon Linux 2023 (AL2023)**

- AWS hardens and maintains Amazon Linux
- Amazon Linux often has direct support or is the only choice for specific AWS services as the underlying OS.
- AWS provides better technical support via AWS Support for Amazon Linux distributions vs non-AWS OS distributions.

The **yum package manager** is used to install packages.

```
yum install iptables
```

The improved yum package manager called “dnf” is available on Amazon Linux 2023



On June 30, 2024, CentOS Linux 7 will reach End of Life (EOL).

Amazon Linux 2 is strongly tied to Cent OS 7, so migrate to Amazon Linux 2023

EC2 – Amazon Linux Extras

Amazon Linux Extras is a feature of Amazon Linux 2 that provides **a way for users to install additional software packages** that are not included in the default Amazon Linux 2 distribution.

```
amazon-linux-extras list  
sudo amazon-linux-extras enable redis6  
sudo yum install redis
```

AL2 was packaged with significantly fewer packages than AL1 making Amazon Linux Extras vital
AL2023 has significantly more packages so Amazon Linux Extras is not available for AL2023

EC2 – Amazon Linux 2 and EPEL

Extra Packages for Enterprise Linux (EPEL) is a special interest group (SIG) from the Fedora Project that provides a set of additional packages for RHEL (and CentOS, and others) from the Fedora sources.

To **install EPEL** you can use Amazon Linux Extras on AL2

```
sudo amazon-linux-extras install epel -y
```

Then install extended packages per usual.

While Amazon Linux Extras can be used to install packages if you need a broader range of packages you can use EPEL7.



Amazon Linux 2 has **high compatibility** with CentOS7 and can support EPEL7 packages.

EC2 – AL2 vs AL2023

AL2

Limited packages but can be extended with Amazon Linux Extras and RHEL7

Python 2.7

Yum as the package manager

Security Enhanced Linux (SELinux) disabled by default

OpenSSL 1.0.2

Sourcing CentOS7 Stream

gp2 volumes by default

Cronie installed by default to have crontab

OpenJDK

AL2023

Contains thousands of packages for use

Python 3

Dnf as the package manager

SELinux enabled by default

OpenSSL 3

Source CentOS9 Stream

gp3 volumes by default

Cronie not installed by default

Amazon Corretto